

Conexão Solidária — Justificativa Técnica

Escolha do banco de dados e do broker de mensageria para o MVP da plataforma de doações

1. Banco de dados: PostgreSQL

O domínio do desafio (campanhas, doadores e doações) é fundamentalmente relacional: entidades com campos obrigatórios, restrições de unicidade (e-mail e CPF do doador), e uma operação sensível a corrida — a soma do valor arrecadado de uma campanha, feita de forma concorrente por múltiplas doações processadas pelo worker. Essas características pedem um banco com garantias ACID fortes e suporte nativo a transações, o que exclui bancos NoSQL eventualmente consistentes como escolha padrão.

Motivos específicos da escolha:

- **Integridade transacional:** o UPDATE que soma o valor arrecadado (`UPDATE campanhas SET "ValorTotalArrecadado" = "ValorTotalArrecadado" + @valor`) precisa ser atômico mesmo sob concorrência — é uma operação nativa e segura em qualquer banco relacional com MVCC, sem precisar de lógica de merge/reconciliação como seria necessário em um banco eventualmente consistente.
- **Constraints declarativas:** unicidade de e-mail e CPF do doador são impostas pelo próprio banco (índices únicos), não só na aplicação.
- **Maturidade do ecossistema .NET:** o Npgsql (driver/EF Core provider) é maduro e já era o padrão usado nos projetos de referência internos que serviram de base para este MVP, reduzindo risco e tempo de setup.
- **Custo operacional:** um único motor de banco atende tanto a aplicação quanto o Zabbix (que também roda sobre Postgres, em um banco separado no mesmo servidor), evitando subir um segundo motor de banco só para a stack de observabilidade.

2. Broker de mensageria: RabbitMQ (em vez de Kafka)

O desafio permite RabbitMQ ou Kafka para a comunicação assíncrona entre a API e o worker de doações. A escolha recaiu sobre o RabbitMQ pelo perfil de uso real do cenário: um único tipo de evento (`DoacaoRecebidaEvent`), publicado a um volume baixo (uma mensagem por doação), consumido por um único consumidor lógico, sem necessidade de replay de histórico nem de múltiplos consumer groups lendo o mesmo tópico de formas independentes.

Motivos específicos da escolha:

- **Baixo throughput, sem necessidade de log de eventos:** o Kafka se paga quando há alto volume, múltiplos consumidores lendo o mesmo stream de formas diferentes, ou necessidade de reprocessar o histórico completo. Nenhum desses cenários existe aqui — o Kafka adicionaria complexidade operacional (Zookeeper/KRaft, partições, tuning de consumer group) sem benefício mensurável para este MVP.

- **Modelo de fila simples resolve o requisito:** uma exchange do tipo `direct` com uma fila vinculada por routing key (`doacao-recebida`) já garante a entrega assíncrona e desacoplada exigida pelo desafio, com confirmação de publicação (`publisher confirms`) e reentrega em caso de falha (`nack + requeue`).
- **Reuso de código já existente e testado:** o cliente RabbitMQ (`ConexaoSolidaria.Shared.RabbitMQ`) foi adaptado de uma biblioteca interna já validada em produção em outro sistema, com publisher, consumer, retry de conexão e logging estruturado prontos — construir o equivalente para Kafka do zero consumiria tempo do hackathon sem agregar valor ao requisito avaliado.
- **Observabilidade mais simples:** o RabbitMQ Management UI expõe filas, mensagens em trânsito e taxa de consumo prontos para o vídeo de demonstração, sem precisar de ferramentas adicionais como Kafka UI/AKHQ.

3. Resumo comparativo

Critério	PostgreSQL	Alternativa considerada
Consistência	ACID forte, essencial para saldo de doações	NoSQL eventual — descartado
Broker	RabbitMQ — fila simples, baixo overhead	Kafka — overhead sem ganho real de throughput
Operação	Um motor de banco para app + Zabbix	MySQL extra só para Zabbix — descartado
Reuso	Cliente RabbitMQ já pronto e testado	Cliente Kafka construído do zero

Documento produzido para o hackathon "Conexão Solidária" (pós-graduação em Arquitetura de Sistemas). Repositórios do projeto: github.com/marcarinivini/conexao-solidaria*